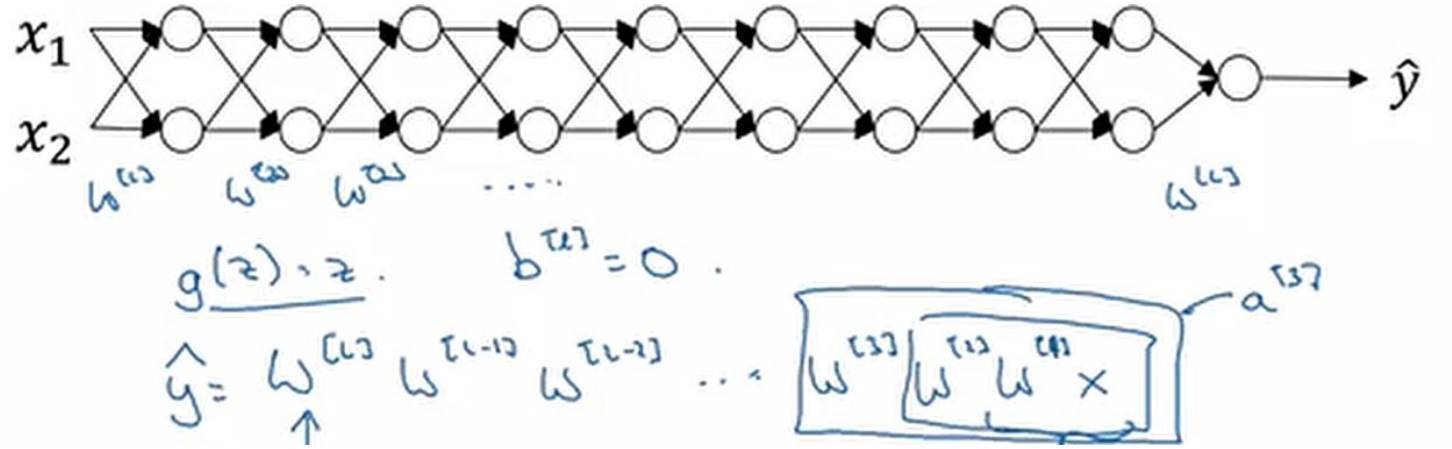
Initialization

미래연구소 16기 7주차

1. zero initialization 0) gradient vanishing/exploding

쉬운 이해를 위해 activation과 bias가 없다고 가정



1> w가 계속해서 작으면 gradient vanishing
2> w가 계속해서 크면 gradient exploding / ex> (1.5)^20 = 3325

=> w의 값을 적절하게 설정할 필요 있다.
(하지만 random initialization으로는 부족 why? 11강에서 이어집니다.)

a가 symmetric이면
dz가 symmetric인 이유는
이전에 식으로 증명

1. zero initialization 1) 문제: row symmetric

$$W^{[l]} = \begin{bmatrix} 0 & 0 \\ 0 & 0 \end{bmatrix}$$

W[l]이 모두 0이면,
g(W[l]X + b) = A[l]의 행들의 원소가 모두 같은
row symmetric한 현상이 생깁니다.

$$a_1^{[l]} = a_2^{[l]}$$ row symmetric

(column은 b를 다르게 설정하면 symmetric 깰 수 있지만 row symmetric은 안 깨집니다.)

$$dz_1^{[l]} = dz_2^{[l]}$$ row symmetric

$$dW^{[l]} = \frac{1}{m} dZ^{[l]} A^{[l-1]T}$$ row symmetric

$$W^{[l]} = W^{[l]} - \alpha dW^{[l]}$$ row symmetric

node를 여러개 설정했음에도

node가 1개인 효과를 갖는다.

1. zero initialization 2) 문제: vanishing gradient

이전에 식들을 통해 증명했습니다.

$$dW^{[l]} = \frac{1}{m} dZ^{[l]} A^{[l-1]T}$$

$$W^{[l]} = W^{[l]} - \alpha dW^{[l]}$$

$$dZ^{[l]} = dA^{[l]} * g'(Z^{[l]})$$

$$dA^{[l-1]} = \frac{\partial \mathcal{L}}{\partial A^{[l-1]}} = W^{[l]T} dZ^{[l]}$$

W[l] = 0이 되면,

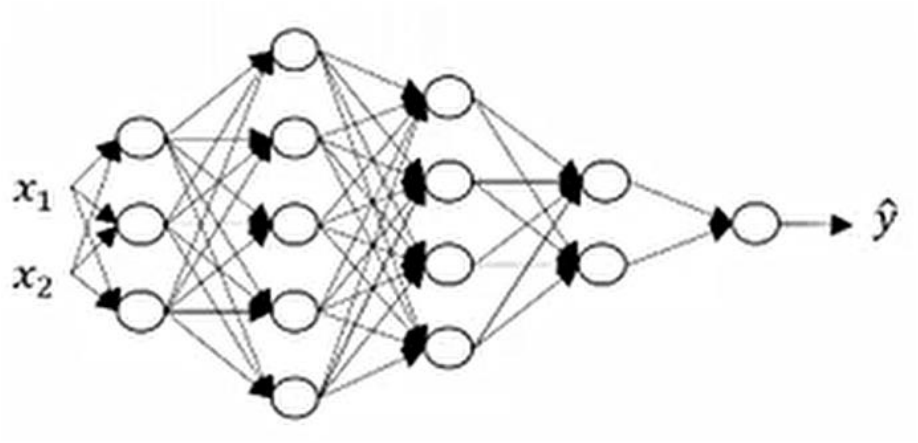
dW[l] = 0이 되는 문제가 생긴다.

(정확히는 W[l+1] =0 => dW[l] = 0)

1. zero initialization 3) 해결: np.random.randn*0.01

$$W^{[l]} = np.random.randn(shape) \quad * \ 0.01$$

2. Random Initialization의 문제점 0) 용어 소개



1> fan-in: 들어오는 layer의 unit 개수

=> n[l-1]

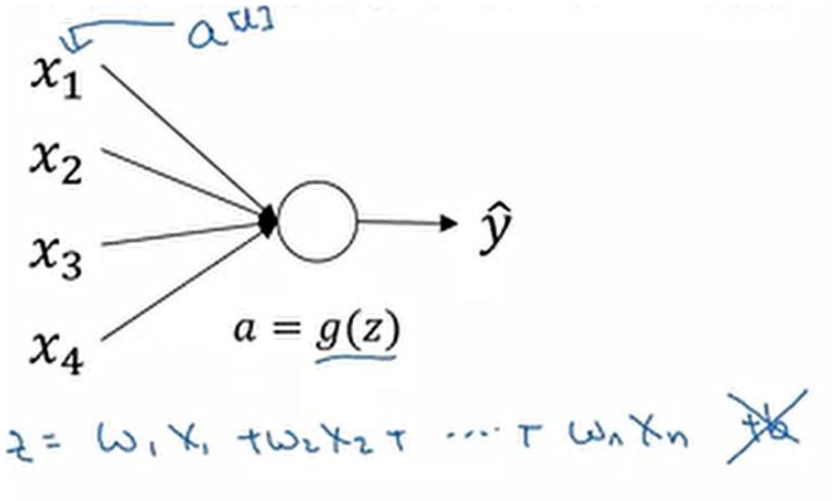
2> fan-out: 현재 layer의 unit 개수

=> n[l]

2번째 layer의 fan-in?

4번째 layer의 fan-out?

2. Random Initialization의 문제점 1) 문제점



fan-in이 길수록

=> 출력되는 fan-out의 값이 커진다.

=> gradient exploding

fan-in이 짧을 수록

=> 출력되는 fan-out의 값이 작아진다.

=> gradient vanishing

Weight Initialization: Activation statistics

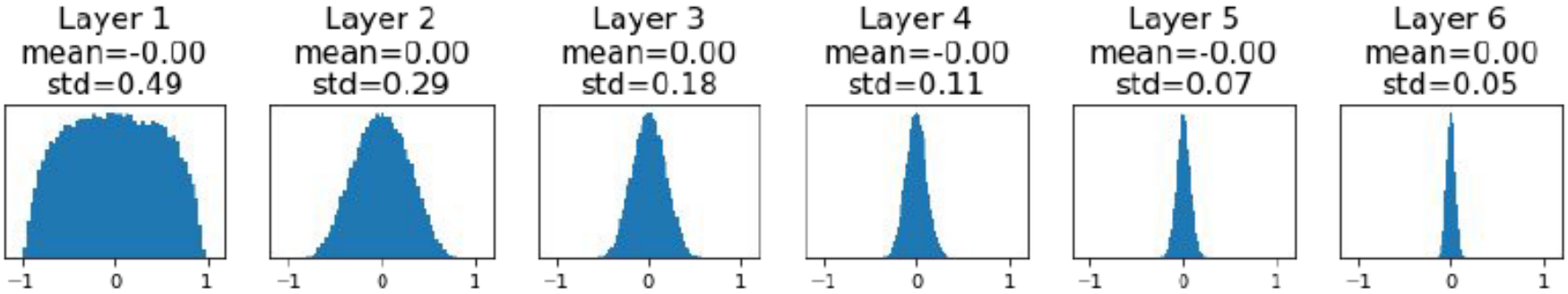
```
dims = [4096] * 7
hs = []
x = np.random.randn(16, dims[0])
for Din, Dout in zip(dims[:-1], dims[1:]):
    W = 0.01 * np.random.randn(Din, Dout)
    x = np.tanh(x.dot(W))
    hs.append(x)
```

Forward pass for a 6-layer net with hidden size 4096

All activations tend to zero for deeper network layers

Q: What do the gradients dL/dW look like?

A: All zero, no learning =(



layer별 output의 분포가 0에 가까워서 gradient vanishing이 일어날 가능성이 높다.

Weight Initialization: Activation statistics

```
dims = [4096] * 7
hs = []
x = np.random.randn(16, dims[0])
for Din, Dout in zip(dims[:-1], dims[1:]):
    W = 0.05 * np.random.randn(Din, Dout)
    x = np.tanh(x.dot(W))
    hs.append(x)
```

Increase std of initial weights from 0.01 to 0.05

What will happen to the activations for the last layer?

그래서 W의 초기화 상수를 키우면?

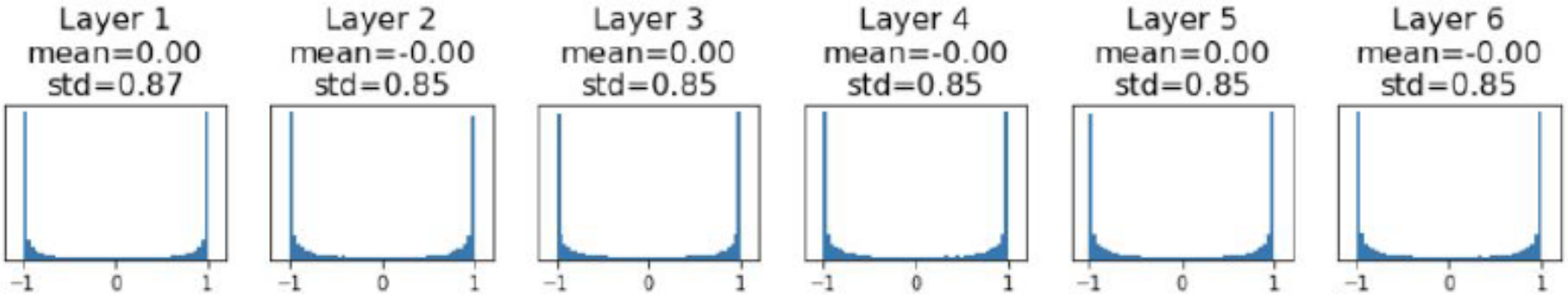
Weight Initialization: Activation statistics

```
dims = [4096] * 7
hs = []
x = np.random.randn(16, dims[0])
for Din, Dout in zip(dims[:-1], dims[1:]):
    W = 0.05 * np.random.randn(Din, Dout)
    x = np.tanh(x.dot(W))
    hs.append(x)
```

Increase std of initial weights from 0.01 to 0.05

All activations saturate

Q: What do the gradients look like?



개선이 된다.

매번 다른 상수를 고르기 어렵다.

fan-in이 클수록 초기화 상수를 작게 설정하면 된다.

해결: fan-in에 반비례한 초기화 상수(이전에는 0.01)를 사용해야 한다.

3. Xavier Initialization 1) 특징

1> glorot initialization이라고도 부름 (tensorflow에서는 이 용어를 사용)

2> 보편적으로 잘 된다.

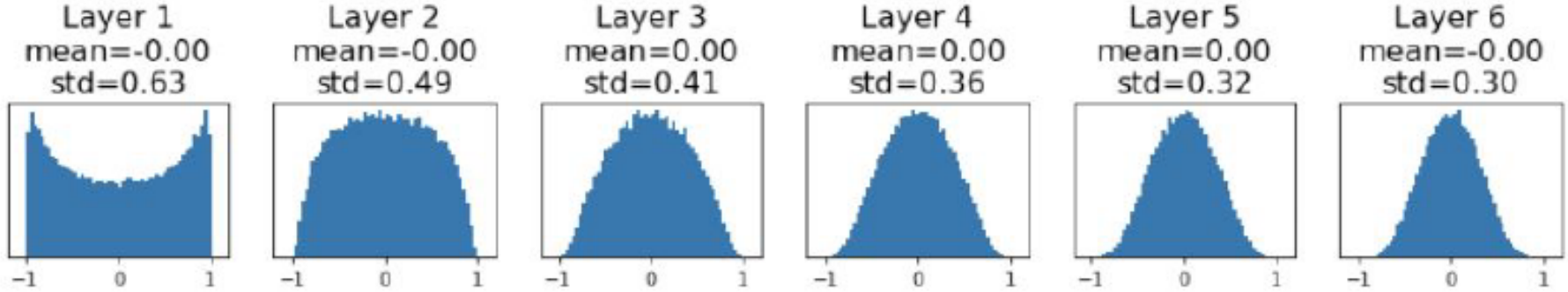
3> 현재는 fan-in과 fan-out을 모두 고려하는 방식을 선택

Weight Initialization: “Xavier” Initialization

```
dims = [4096] * 7
hs = []
x = np.random.randn(16, dims[0])
for Din, Dout in zip(dims[:-1], dims[1:]):
    W = np.random.randn(Din, Dout) / np.sqrt(Din)
    x = np.tanh(x.dot(W))
    hs.append(x)
```

“Xavier” initialization:
std = 1/sqrt(Din)

“Just right”: Activations are nicely scaled for all layers!



Glorot and Bengio, “Understanding the difficulty of training deep feedforward neural networks”, AISTAT 2010

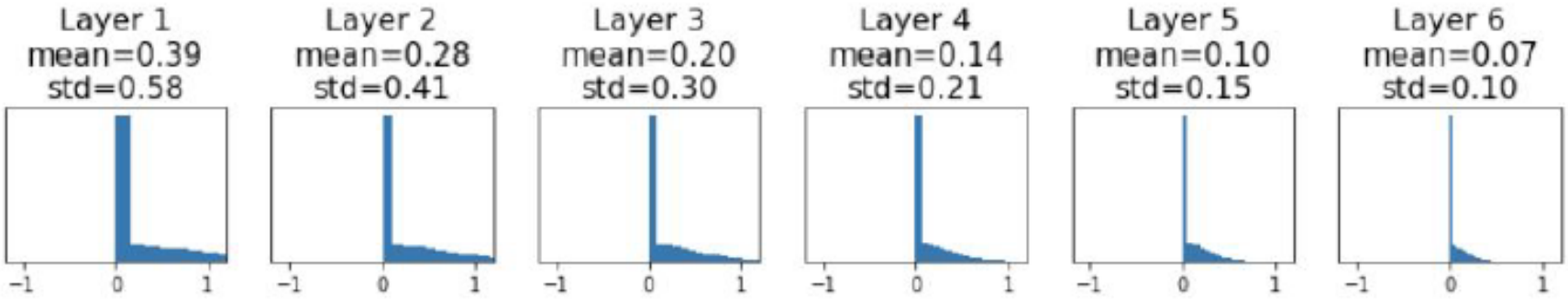
Weight Initialization: What about ReLU?

```
dims = [4096] * 7
hs = []
x = np.random.randn(16, dims[0])
for Din, Dout in zip(dims[:-1], dims[1:]):
    W = np.random.randn(Din, Dout) / np.sqrt(Din)
    x = np.maximum(0, x.dot(W))
    hs.append(x)
```

Change from tanh to ReLU

Xavier assumes zero centered activation function

Activations collapse to zero again, no learning =(



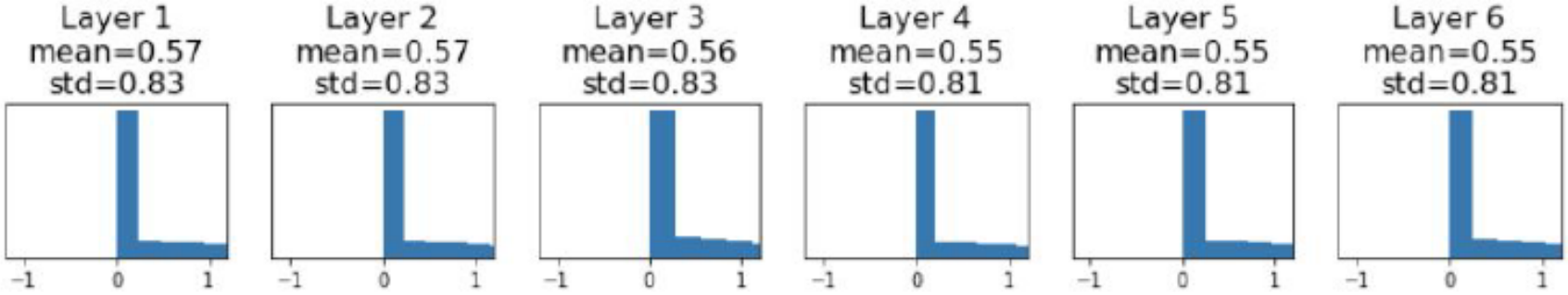
하지만 ReLU를 쓰면 layer별 output이 0 주변에 과도하게 몰릴 수 있다.

Weight Initialization: Kaiming / MSRA Initialization

```
dims = [4096] * 7
hs = []
x = np.random.randn(16, dims[0])
for Din, Dout in zip(dims[:-1], dims[1:]):
    W = np.random.randn(Din, Dout) * np.sqrt(2/Din)
    x = np.maximum(0, x.dot(W))
    hs.append(x)
```

ReLU correction: std = sqrt(2 / Din)

"Just right": Activations are nicely scaled for all layers!



He et al, "Delving Deep into Rectifiers: Surpassing Human-Level Performance on ImageNet Classification", ICCV 2015

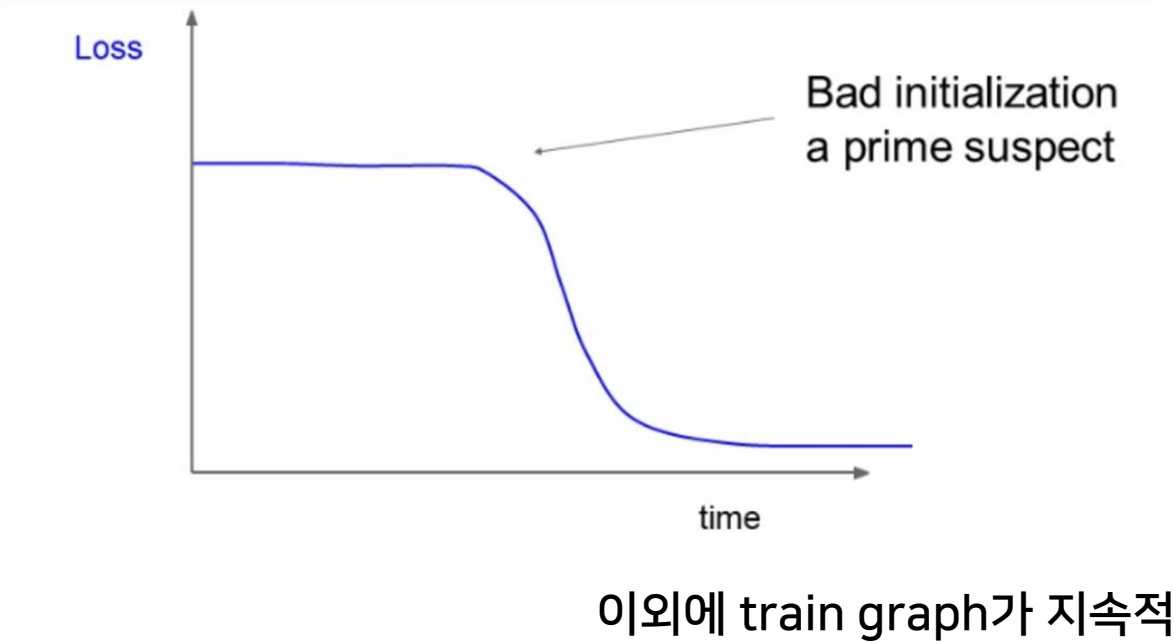
그래서 √2를 곱하면 분포를 좀 더 1쪽으로 넓힐 수 있다. (He)

4. He Initialization 1) 특징

1> fan-in만 고려하는 방식

2> ReLU계열에 특화

5. Bad Initialization



이외에 train graph가 지속적으로 감소하지 않는 경우
initialize 방법을 바꿀 필요가 있습니다.

출처: CS231n 2017 winter

무엇이 정답일지는 data의 형태에 따라 다릅니다.

두 가지 방식은 다 사용해보시고

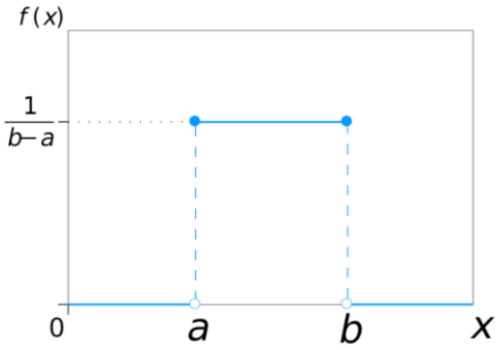
학습에 지장이 안 생기는 방식을 사용합니다.

뒤는 그리 중요하지는 않습니다. 참고만 하세요.

통계 용어

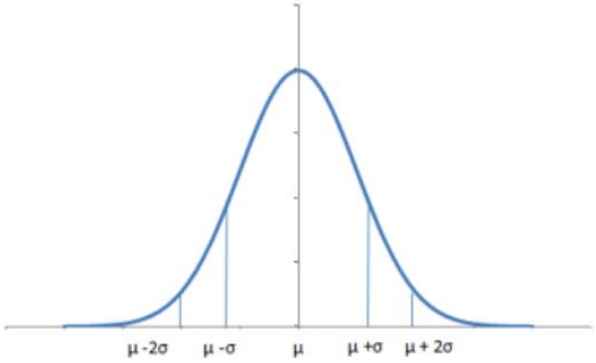
1) Uniform distribution

1> 해당 범위 내에서 모든 값이 나올 확률이 같다.
2> 쉽게 생각해서 주사위 던지기



2) Normal distribution

1> 고등학교 때 배운 정규분포
2> 윷놀이를 연상하면 쉽습니다.
(도보다는 중간의 개와 걸이 더 잘 나옵니다.)



4. Xavier Initialization 2) glorot_normal

$$W \backsim N(0, Var(W))$$

$$Var(W) = \frac{2}{n_{in} + n_{out}}$$

4. Xavier Initialization 3) glorot_uniform

$$W \backsim U\left(-\sqrt{\frac{6}{n_{in}+n_{out}}}, +\sqrt{\frac{6}{n_{in}+n_{out}}}\right)$$

5. He Initialization 2) he_normal

$$W \backsim N(0, Var(W))$$

$$Var(W) = \frac{2}{n_{in}}$$

5. He Initialization 3) he_uniform

$$W \backsim U(-\sqrt{\frac{6}{n_{in}}}, +\sqrt{\frac{6}{n_{in}}})$$